Candidate Onboarding with CV Upload — System Design

Dental recruitment platform · Rails module · v1 (MVP)

July 2026

0. How to read this

This is an engineering design for the PRD you shared. Context assumed from your answers: an **existing Rails platform** (this is a new module, not a new app), **low volume (<100 CVs/day)**, and I've been asked to **recommend the CV-parsing approach**. Where the PRD left a decision open, I've made one and marked the trade-off. Section 15 lists what I'd revisit as volume grows.

The one-line summary: it's a **two-controller Hotwire flow backed by a Postgres data model, with CV parsing pushed to a background job that calls an LLM with a strict JSON schema, then maps the result onto editable, reference-data-driven form fields.** Nothing here needs Kafka, microservices, or a separate parsing service — at your scale that would be over-engineering.

---

1. Requirements

Functional (what it does)

- Two-step flow: (1) upload CV, (2) review + complete a prefilled profile.
- Accept PDF / DOC / DOCX, configurable max size (default 25 MB).
- Extract candidate data from the CV and prefill the form; never invent data.
- Let the candidate edit everything, add/remove education and work-experience records, pick skills scoped to their job function, and save.
- Conditional fields driven by job function and employment type.
- Store the CV privately, notify recruiters on completion, optionally fire a webhook.

Non-functional (the constraints that shape the design)

Concern	Target at v1	Why it matters here
Volume	<100 CVs/day (~a few/hour peak)	Lets us stay single-Postgres, no autoscaling parsing tier
Parse latency	5–30 s acceptable	Async job + loading state; form must not block on it
Availability	Standard web app (no five-nines)	Recruitment onboarding, not payments
Privacy	CVs are PII; not publicly accessible	Signed URLs, authz, consent, EU/GDPR posture

Languages	Dutch + English min., EU dental formats	Drives the parser choice
Maintainability	Reference data admin-editable	Job functions/skills/regions change over time

Constraints

- Ruby on Rails, existing app -> reuse its User, auth, admin, and CI.
- Small team implied -> favour boring, Rails-native tools over bespoke infra.

---

2. Key architectural decisions (with trade-offs)

D1 — CV parsing: LLM with structured (JSON-schema) output. (recommended)

The parser has to turn free-form, multi-language, multi-format dental CVs into ~15 typed fields plus repeated education/experience/skill records, and flag uncertainty. Three realistic options:

Option	Fit for this problem	Cost / effort	Data residency
LLM + structured output (Claude/GPT, JSON schema)	**Excellent** — handles NL+EN, varied layouts, semantic mapping ("tandarts" -> General dentist), summaries, per-field confidence	Low build, per-call API cost (trivial at <100/day)	Data leaves your servers (mitigable: EU endpoint / zero-retention / redact before send)
Dedicated resume-parsing API (Affinda, Sovren/Textkernel, HireAbility)	Good for generic CVs; **weaker on dental-specific taxonomy** (BIG status, function categories) without custom config	Medium (integration + per-doc fee)	Third-party, same residency concern
Self-hosted / OSS (spaCy, regex, layout models)	Poor without heavy investment; brittle across formats/languages	High build + ongoing tuning	Best — stays in-house

Why LLM wins at your scale: the hard part is *semantic* — mapping messy phrasing to your fixed job categories and skill groups, in two languages, and admitting when it doesn't know. LLMs do exactly this, the volume makes API cost negligible, and the build is small. A dedicated parser adds a taxonomy-mapping problem you'd still solve yourself; OSS adds a research project.

Guardrails that make it safe (these matter more than the model choice): - Send **extracted text**, not the raw file, to a model configured for **zero data retention**; prefer an **EU-region endpoint** for GDPR. - Constrain output to a **strict JSON schema** (tool/function-calling or response-format). The schema is the contract between the parser and the form. - Prompt rule: *"Only extract values present in the text. If absent, return null. Never guess."* Every field carries a confidence and source_span. - **You still never trust it** — the candidate reviews and confirms everything before save (this is a PRD requirement and also the mitigation for hallucination).

> Escape hatch designed in from day one: parsing is best-effort. If it fails, times out, or the candidate abandons it, they complete the form manually. The product does not depend on the parser being correct.

D2 — Parsing runs in a background job, not the request.

A 5–30 s LLM call cannot sit in a web request. Upload returns immediately; the job (ParseCandidateCvJob) does extract -> LLM -> map -> persist draft, and the UI polls/streams the result. This is also explicitly in the PRD.

D3 — Queue backend: GoodJob (Postgres-backed) over Sidekiq/Redis. (for v1)

At <100 jobs/day you don't need Redis. GoodJob runs on the Postgres you already have — one fewer piece of infra to run, secure, and back up. It supports retries, scheduling, and a dashboard. **Revisit** (-> Sidekiq) only if job volume or fan-out grows. *(If the platform already runs Sidekiq, just reuse it — don't add GoodJob.)*

D4 — Frontend: server-rendered Hotwire (Turbo + Stimulus).

Conditional show/hide and the education/experience repeaters are the only "interactive" parts. Hotwire is Rails-native, keeps validation authoritative on the server, and avoids standing up a separate SPA for one form. Turbo Streams also cleanly handle the "parsing -> prefilled form appears" transition and the polling. A React SPA would be more machinery than this screen warrants.

D5 — Form options live in the database, not hardcoded.

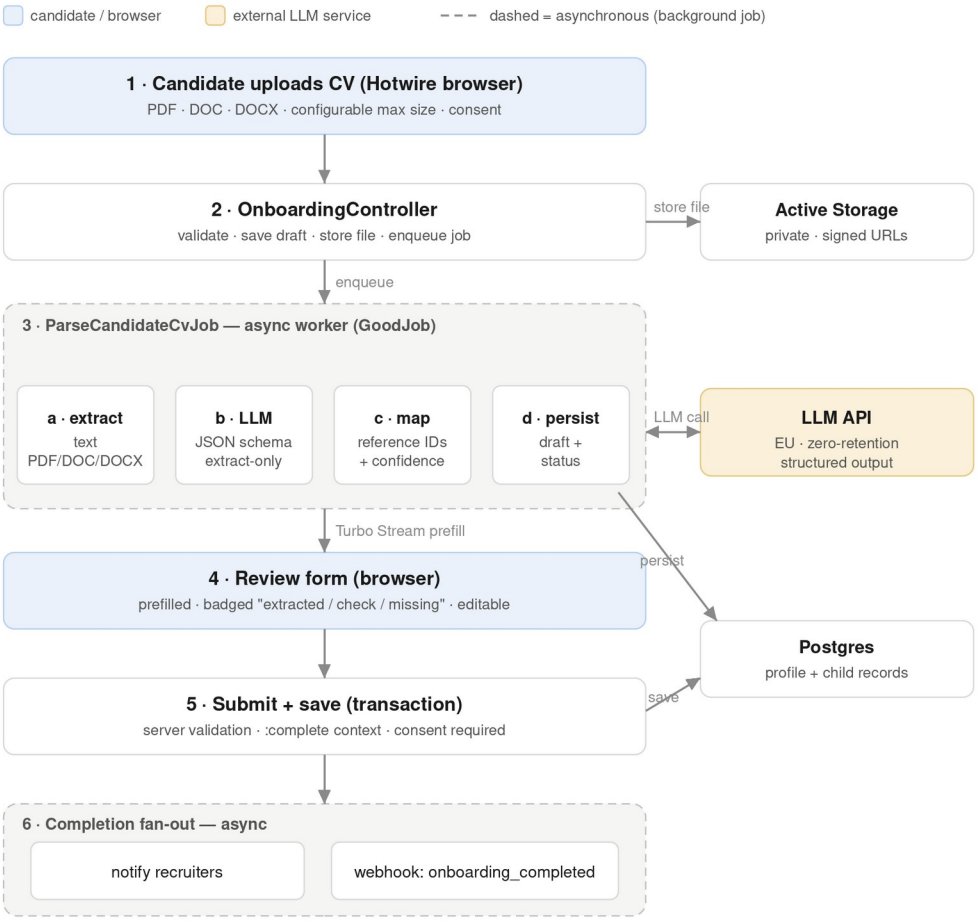
Job functions, regions, employment types, skills (grouped by function), languages, working days -> seeded reference tables, editable in the existing admin. The PRD calls for this, and it's what lets the parser map onto stable IDs and lets recruiters evolve the taxonomy without a deploy.

D6 — Draft persisted before parsing completes.

Create the CandidateProfile (status draft) and CandidateDocument at upload time. This is what makes "candidate closes the page during parsing" recoverable — they return via a resume link and the upload + parse result are already there. Nothing is lost because the parse result is written to the draft, not held in the browser.

---

3. High-level architecture



Architecture diagram

Component responsibilities - **OnboardingController** — thin. Validates file type/size, creates the draft, kicks off the job, renders the review screen in a "processing" state. - **ParseCandidateCvJob** — the whole parsing pipeline (§5). Idempotent and retryable; updates parsing_status. - **Review form** — reads the draft + parse result, renders every section, marks fields "Extracted" / "Please check" / "Missing". - **Reference tables** — the source of truth for selectable options and the parser's mapping targets.

---

4. Data model

Building on the PRD's suggestion, with the scalar fields and reference tables it implies.

```
User 1---1 CandidateProfile 1---* CandidateDocument (has_one_attached :file)
                            | 1---* Education
                            | 1---* WorkExperience
                            | 1---* CandidateSkill *---1 Skill *---1 FunctionGroup
                            | 1---* CandidateLanguage *---1 Language
```

| *---1 JobFunction (desired function)
| *---* Region (preferred regions, join)
| *---* EmploymentType (desired types, join)

CandidateProfile (main aggregate) — key columns

- Personal: first_name, last_name, email, phone (E.164), city, country, ...
- Job prefs: job_function_id, max_travel_time, transport_types (array/enum), search_status (enum: active/passive/inactive), reason_for_looking
- Employment/comp: desired_gross_salary, desired_percentage, average_daily_revenue, big_status (enum), big_number, years_experience
- Availability: working_days (array of enum), available_from (date), notice_period
- Additional: motivation, internal_notes, professional_summary
- Meta: status (enum: draft -> completed), consent_accepted_at, completed_at
- Regions/employment types via join tables (multi-select).

  Design note on nullability: at draft stage almost everything is optional; the PRD's "required" fields are enforced **only on the completed transition** (see §7 conditional validation). Don't put NOT NULL on business-required fields — enforce them in a validation context, or you can't save a partial draft.

CandidateDocument

belongs_to :candidate_profile, has_one_attached :file, plus: document_type (enum, cv), original_filename, content_type, file_size, parsing_status (enum: pending/processing/completed/failed), parsed_at, parsed_payload (jsonb) <- the raw structured LLM result, **recruiter/admin only, never shown to the candidate** (PRD: don't expose raw parser output), parse_error (text).

Education / WorkExperience

Separate rows (PRD is explicit: multiple employers/schools). Standard fields per the PRD, each with an optional source (cv | manual) so the UI can badge extracted rows.

Skills

- Skill belongs_to :function_group — drives "show skills for the selected function".
- CandidateSkill join with source (cv/manual) and status.
- **Unknown CV skills** -> stored as free-text suggestions (CandidateSkillSuggestion, or a raw_label on CandidateSkill with skill_id: nil) flagged for recruiter review, per the PRD. They never pollute the canonical Skill list automatically.

Reference tables (seeded, admin-editable)

JobFunction, Region, EmploymentType, Skill (+FunctionGroup), Language. Enums (search_status, big_status, working_days, transport_types, level) can stay as Rails enums since they're small and stable.

---

5. CV parsing pipeline (deep dive)

Inside ParseCandidateCvJob, four stages, each independently failable:

a. Text extraction — the unglamorous, format-specific part. - **PDF**: pdf-reader gem, or shell out to pdftotext (poppler) which is more robust on real-world PDFs. Scanned/image PDFs -> OCR (Tesseract) is *out of scope for v1*; treat as "little info extracted" and fall back to manual. - **DOCX**: it's a zip of XML — the docx gem or direct document.xml parse. - **DOC (legacy binary)**: the awkward one. No clean pure-Ruby reader; convert with **LibreOffice headless** (soffice --headless --convert-to) or antiword/textract. **Flag:** this adds a system dependency to your worker image — decide early whether DOC is worth it or whether you accept DOCX/PDF only and ask DOC users to re-save. (PRD requires DOC, so budget for LibreOffice in the worker container.)

b. LLM structured extraction — send the text + a JSON schema describing every target field. Instruction set: extract-only, null-when-absent, per-field confidence, detect language, produce an optional professional_summary. Output validated against the schema before use; a schema-invalid response is a soft failure -> retry once, then failed.

c. Normalisation & mapping — turn the model's strings into your IDs: - Fuzzy/canonical match job_function and skills to reference rows (exact -> alias table -> embedding/similarity fallback). Unmatched skills -> suggestions. - Phone -> E.164 (phonelib); dates -> Date; languages -> Language + level; BIG status -> enum. - **Confidence policy:** high-confidence + matched -> prefill normally (badge "Extracted from CV"). Low-confidence or fuzzy match -> prefill but badge **"Please check"**. Absent -> leave empty, badge **"Missing"**.

d. Persist — write onto the draft profile + child records, set parsing_status = completed, parsed_at, store raw parsed_payload. Broadcast a Turbo Stream so the waiting review page swaps from "processing" to the prefilled form.

Idempotency: the job clears prior source: cv prefill before rewriting, so a retry doesn't duplicate education/experience rows.

---

6. Screens, routes, API

Two screens, RESTful-ish nested routes under the profile:

```
GET    /onboarding                    -> step 1, upload screen
POST   /onboarding/documents          -> create draft + upload + enqueue parse
GET    /onboarding/:id/review         -> step 2, review form (processing | ready)
GET    /onboarding/:id/parse_status   -> Turbo/JSON poll (or use Turbo Stream broadcast)
PATCH  /onboarding/:id                -> save/submit final profile
POST   /onboarding/:id/educations     + nested repeater endpoints
DELETE /onboarding/:id/educations/:eid + (Turbo frames; same for work_experiences)
```

Server-rendered HTML + Turbo Streams; no separate JSON API needed for v1. Education/experience use accepts_nested_attributes_for **or** Turbo-frame add/remove rows — the latter is cleaner for dynamic repeaters.

---

7. Conditional form logic

Two layers, and you need **both** — client for UX, server for truth: - **Client (Stimulus):** show/hide BIG fields, salary vs percentage, average daily revenue, and the skill set, based on selected job_function and employment_type. Purely presentational. - **Server (conditional validations):** the same rules re-expressed as validates …, if: -> { … }, run **only in the :complete context** on final submit. E.g. big_number required only when big_status == :registered; salary required only when employment includes an employed type; percentage 0–100; region >=1; working_days >=1.

Encode the function->(fields, skill group) rules as **data, not scattered ifs** — a small JobFunctionRule config (or columns on JobFunction: requires_big, revenue_based, function_group_id) so the clinical/assistant/front-office/manager/technician behaviour is table-driven and matches the reference data the admin edits.

---

8. Privacy & security

- **Private storage:** Active Storage with a non-public service; serve only via **short-expiry signed URLs**. Never a public bucket.
- **Authorization:** Pundit (or existing policy layer) — a CV/profile is viewable by its owning candidate and authorized recruiters/admins only.
- **Consent:** a required checkbox -> persist consent_accepted_at (timestamp, not just a boolean) with the consent text version. Block save without it.
- **Raw parser output** (parsed_payload) is admin/recruiter-only; the candidate UI reads mapped fields, never the JSON.
- **Data minimisation / GDPR:** store only recruitment-relevant data; prefer an **EU LLM endpoint with zero retention**; consider redacting obvious direct identifiers before the LLM call if policy is strict. Plan a deletion path (candidate/CV removal) even if the UI for it is post-MVP.

---

9. Error handling & resilience

Scenario	Behaviour
Unsupported type / too large	Reject at controller (and client) with a clear message before any storage
Upload fails	Candidate retries; no partial draft junk
Text extraction fails (e.g. scanned PDF)	Mark failed, surface “we couldn’t read this — continue manually”

LLM fails / times out / invalid schema	Retry once with backoff; then failed -> manual path
Little info in CV	Prefill what exists, leave rest empty/"Missing" — not an error
Candidate closes page mid-parse	Draft + upload already persisted; resume link restores state
Parser extracted wrong data	Candidate edits before save (core mitigation)
Incomplete submission	Server validation (:complete context) returns field errors

Job retries are bounded (GoodJob retry_on with backoff); a poisoned document goes to failed, never an infinite loop. The UI always has a **"Continue manually"** affordance so a stuck parse never blocks onboarding.

---

10. Completion fan-out

On successful PATCH that transitions draft -> completed, in a transaction: persist profile + children, set completed_at, then enqueue side-effects (don't do them inline): - NotifyRecruitersJob — ActionMailer and/or in-app notification (the noticed gem is a good fit). - CandidateWebhookJob — POST candidate_onboarding_completed (optional for MVP; signed payload, retried). Keep the payload to IDs + summary, not the raw CV.

---

11. Scale & reliability (right-sized for <100/day)

- **One Postgres, one web tier, one or two worker processes.** No sharding, no dedicated parse cluster.
- LLM calls dominate parse latency, not your infra — a single GoodJob worker handles this volume with headroom.
- **Cost:** at <100 CVs/day, LLM parsing cost is on the order of cents-to-low-dollars/day. Not a design driver.
- **Monitoring:** track parsing_status distribution (failure rate is the health metric), job queue latency, and LLM error/timeout rate. Alert if failure rate climbs — that usually means a new CV format or an upstream API issue.
- **Redundancy:** standard web-app posture (managed Postgres backups, retried jobs). Nothing bespoke needed at v1.

---

12. Testing strategy

- **Unit:** conditional validation matrix (function x employment x BIG) — this is where bugs hide; table-test every branch. Mapping/normalisation (phone, dates, skill matching).

- **Job:** ParseCandidateCvJob with the **LLM stubbed** — feed fixed model responses (good, partial, empty, malformed) and assert the draft/state transitions. Never hit the real API in tests.
- **Extraction:** a small corpus of real-ish **NL and EN** dental CV fixtures in PDF/DOCX/DOC -> assert text comes out.
- **System (Capybara):** upload -> processing -> prefilled -> edit -> add/remove records -> validation errors -> save -> recruiter can view.
- **Security:** authz tests that a candidate can't read another's CV; signed-URL expiry.

---

13. Build sequence (suggested)

1. **Data model + reference seeds + admin** — profile, children, reference tables, enums. Nothing works without this.
2. **Manual happy path end-to-end** — upload (store only, no parse) -> review form with all sections, conditional logic, repeaters, validations -> save -> recruiter view. *This alone satisfies most of the PRD and de-risks the parser.*
3. **Parsing pipeline** — extraction -> LLM+schema -> mapping -> prefill + badges, behind the already-working manual form.
4. **Fan-out** — recruiter notification, then webhook.
5. **Hardening** — DOC support decision, failure UX, monitoring, deletion path.

Sequencing parser *after* the manual form means the product is shippable even if parsing slips, and the parser is pure upside on top of a working form.

---

14. What's explicitly out of scope for v1

Per the PRD: completeness score, duplicate detection, automatic BIG validation, AI debug dashboard, multilingual UI, auto job-matching, OCR of scanned CVs, complex legal workflows.

---

15. What I'd revisit as it grows

- **Volume increases** -> GoodJob -> Sidekiq/Redis; multiple workers; rate-limit/queue the LLM.
- **Parser quality plateaus** -> add an alias/synonym table for job functions and skills; consider a fine-tune or a dedicated parser for the extraction stage while keeping LLM for mapping.
- **Strict data residency** -> move extraction fully in-house (self-hosted model) and keep the schema contract unchanged.
- **ATS/CRM features** -> the reference-data + structured-record model is already the right shape; expose a read API and richer webhooks.
- **Scanned CVs common** -> add an OCR stage before extraction.

16. Open questions for the team

1. **DOC (legacy binary):** commit to LibreOffice in the worker image, or restrict to PDF/DOCX for v1?
2. **LLM provider & residency:** is an EU, zero-retention endpoint available/approved, or does policy require fully in-house extraction?
3. **Existing infra:** does the platform already run Sidekiq/Redis (reuse it) or is GoodJob-on-Postgres the lighter path?
4. **Reference data ownership:** who maintains job functions/skills — is there an admin surface already, or does this module add one?
5. **Consent versioning:** do you need to retain *which* consent text a candidate accepted (recommended for GDPR)?